

基于 Kubernetes 实现 DevOps 流程

星环科技

T R A N S W A R P

星环信息科技（上海）股份有限公司

www.transwarp.cn

Copyright©2023 Transwarp. All Rights Reserved.



Contents

- 01 >** DevOps 简介
- 02 >** Kubernetes 简介
- 03 >** DevOps 通用流程介绍
- 04 >** 使用 Kubernetes 实现 DevOps 通用流程
- 05 >** 演示



本书由南京大学软件学院三位资深教师联合行业一线专家编写而成，系统全面地介绍DevOps - 这一互联网时代新型软件开发模式的原理、方法和实践。内容详实、结构清晰、表述浅显易懂，非常适合在校学生学习使用，也可以作为产业界DevOps 初学者学习参考。

01

DevOps 简介

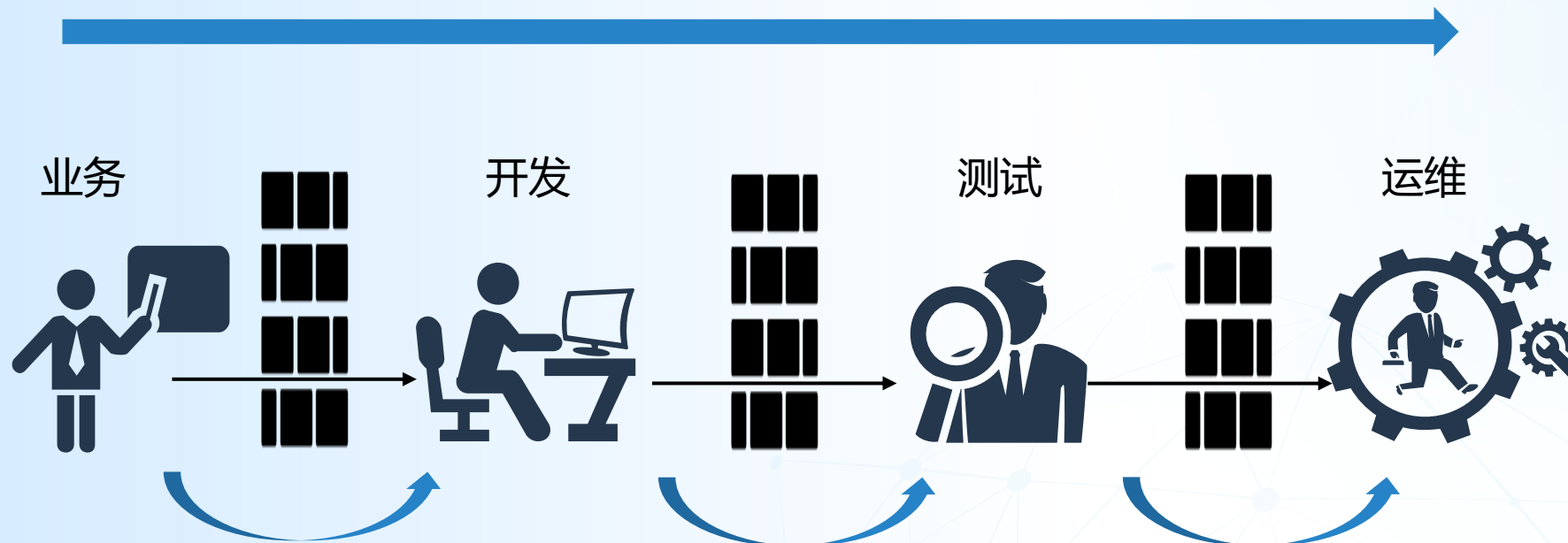
构建明日数据世界

www.transwarp.cn

Copyright©2023 Transwarp. All Rights Reserved.

问题一：Dev 和 Ops 之间的墙阻断了持续交付价值流

价值流被墙阻断



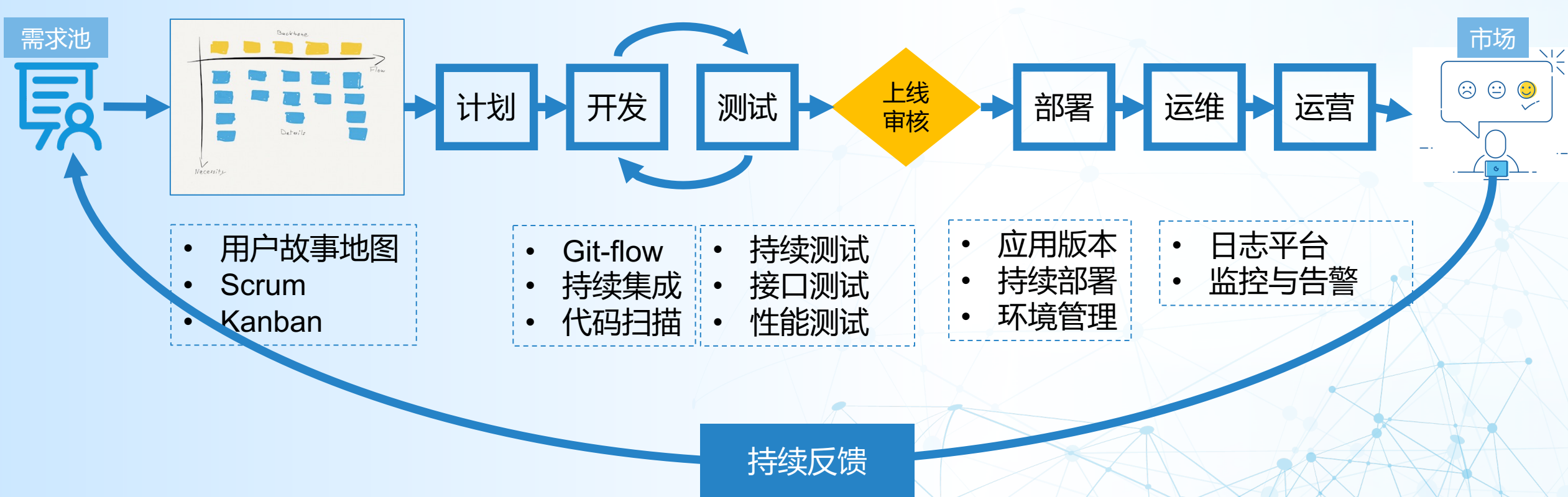
- 需求以文档传递，缺乏沟通
- 需求和计划随意变更
- 项目进度不可控
- 风险不透明

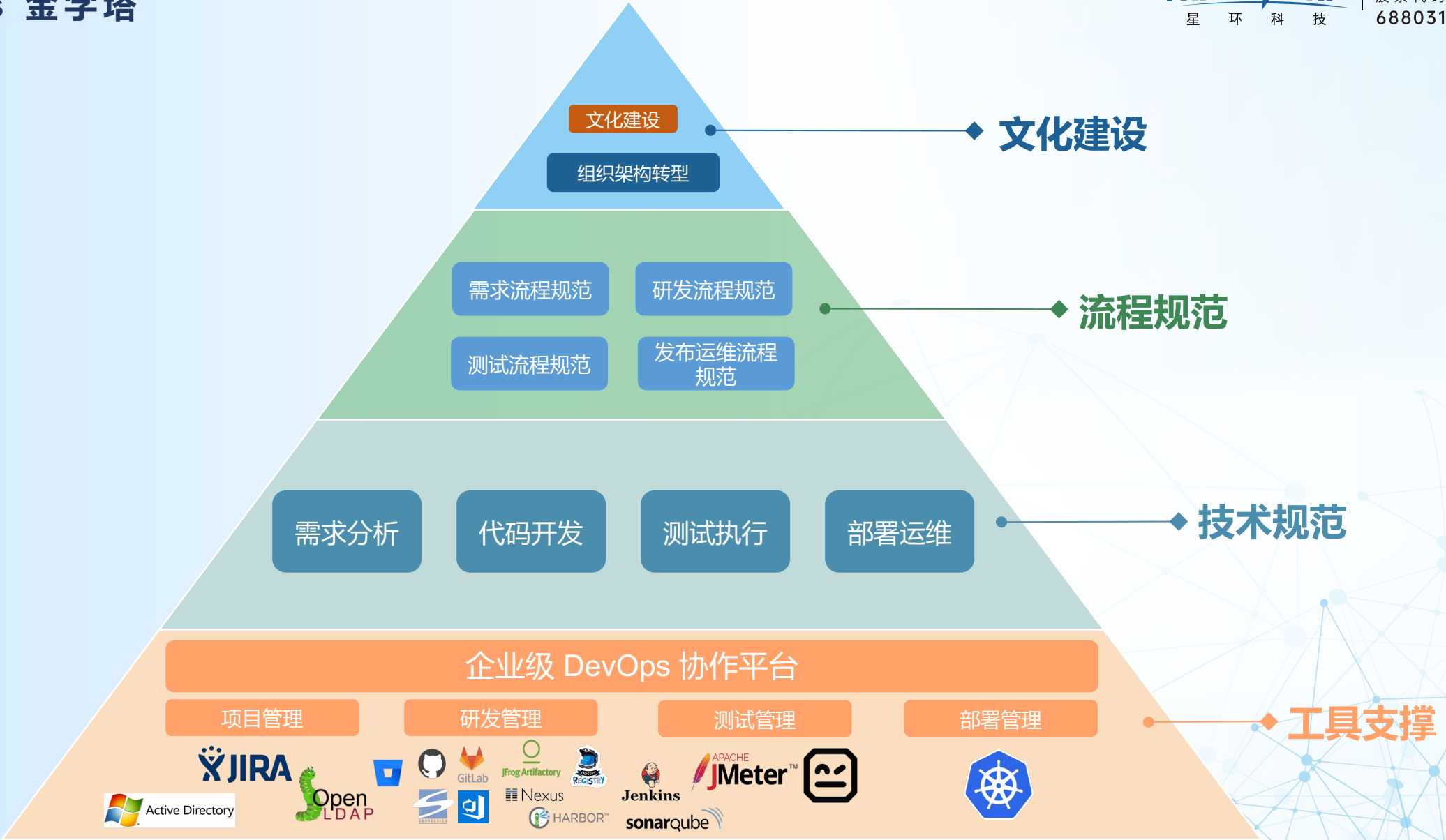
- 功能特性以文档传递，缺乏沟通
- 功能变更影响测试计划
- 频繁修改和打包
- 重复工作占比高

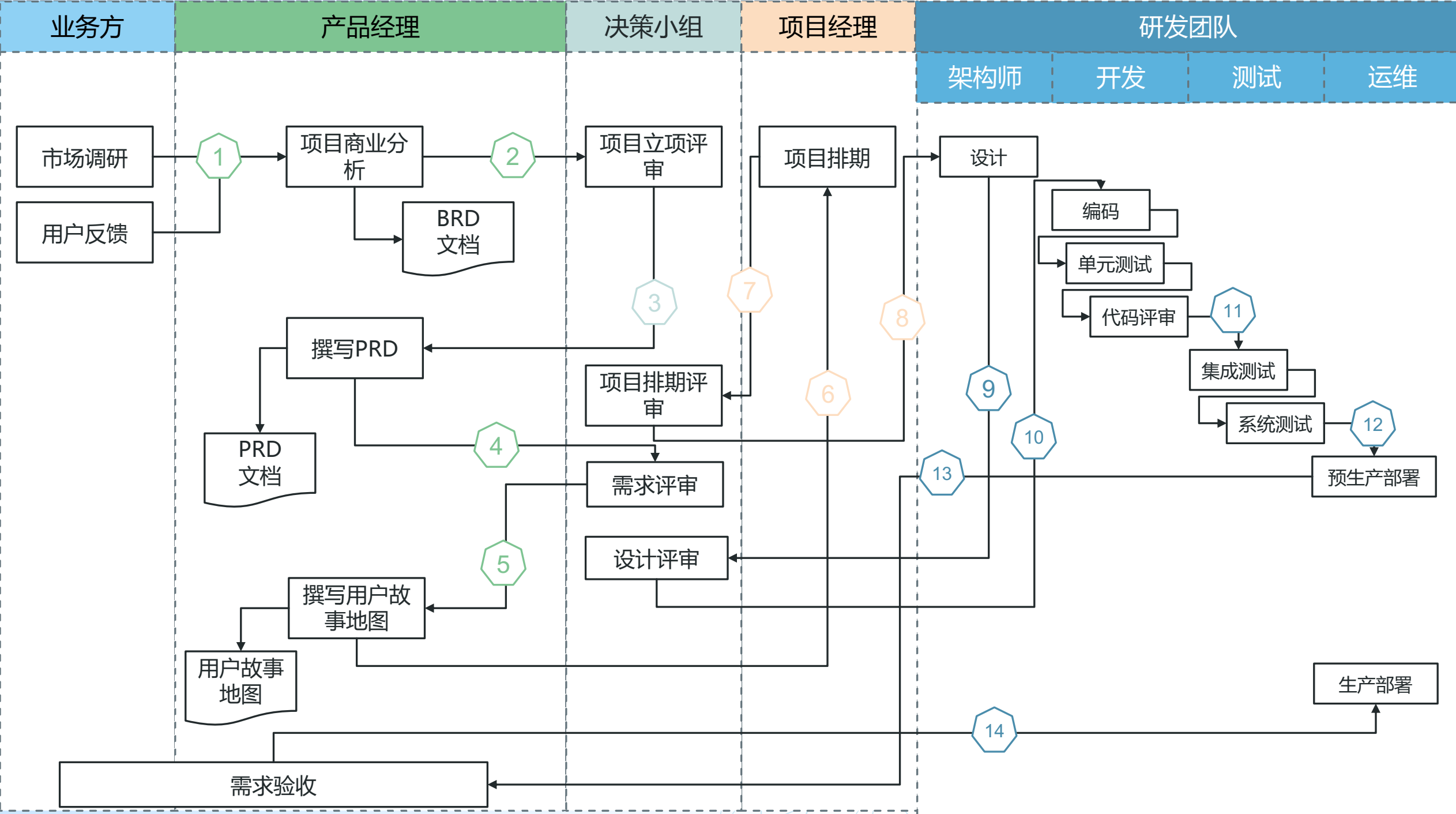
- 频繁发布
- 故障频繁，无法快速回滚
- 发布耗时较长
- 监控告警不及时

问题二：缺少端到端的持续交付流程和平台

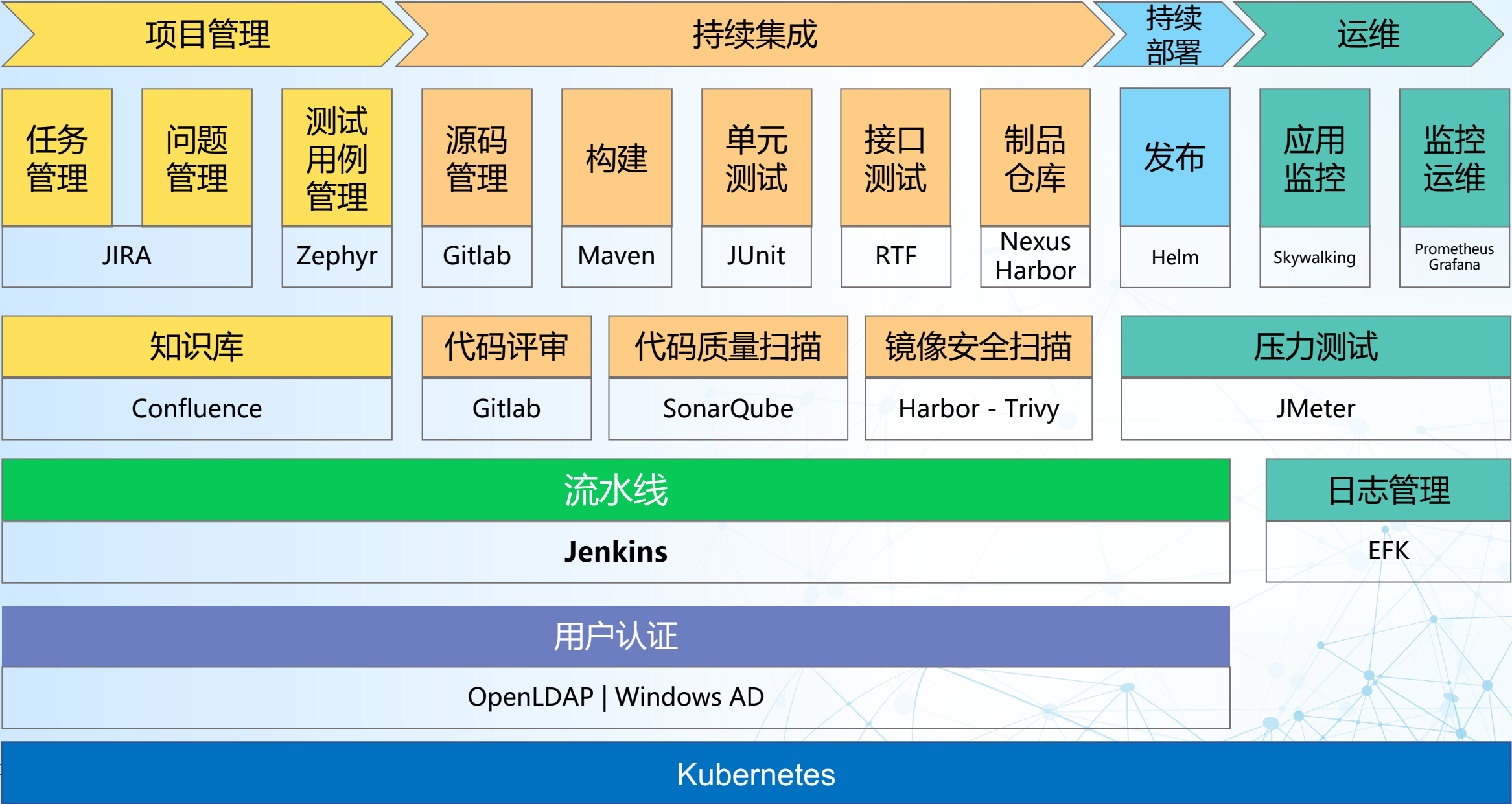
开发、测试和运维使用不同工具完成关心的任务，工具互不关联，软件包在不同工具间转移多依靠人工完成；由于从获取代码、编译、扫描、构建、测试、部署、发布到获得反馈的整个过程中涉及到很多工具的运用，很难有团队能够依靠自身力量在每个环节都做到成熟。







整合与打通 DevOps 工具链



02

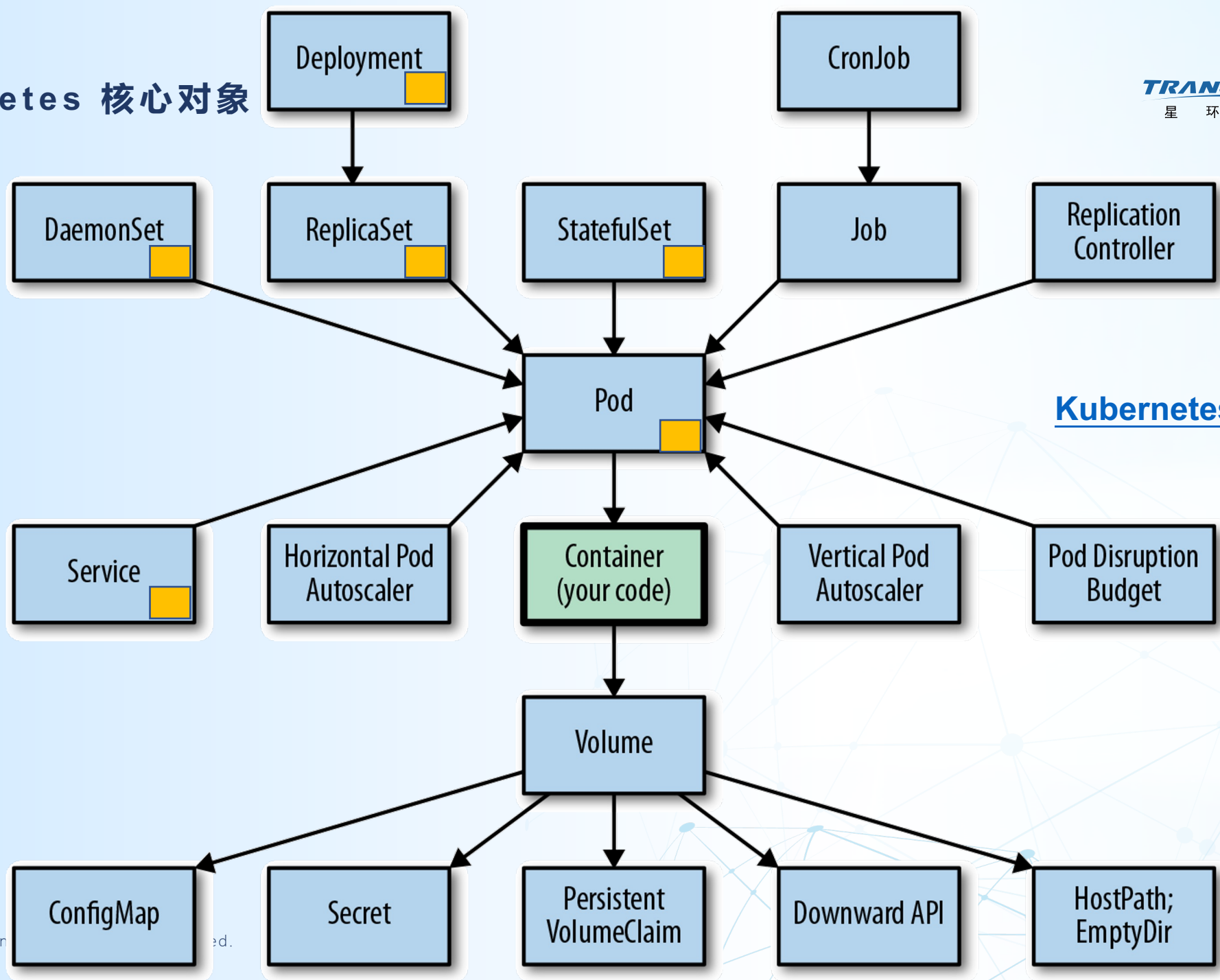
Kubernetes 简介

构建明日数据世界

www.transwarp.cn

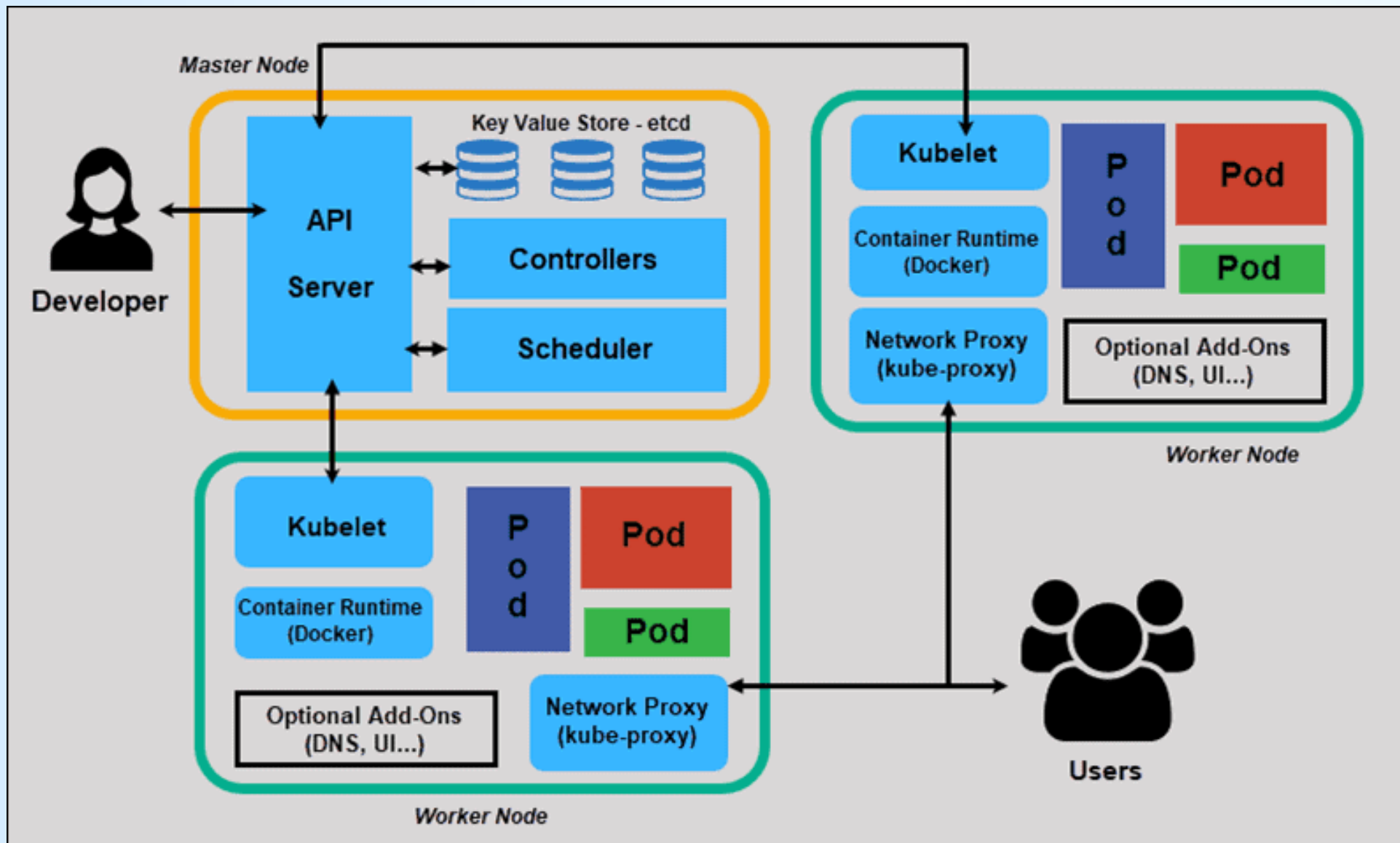
Copyright©2023 Transwarp. All Rights Reserved.

Kubernetes 核心对象



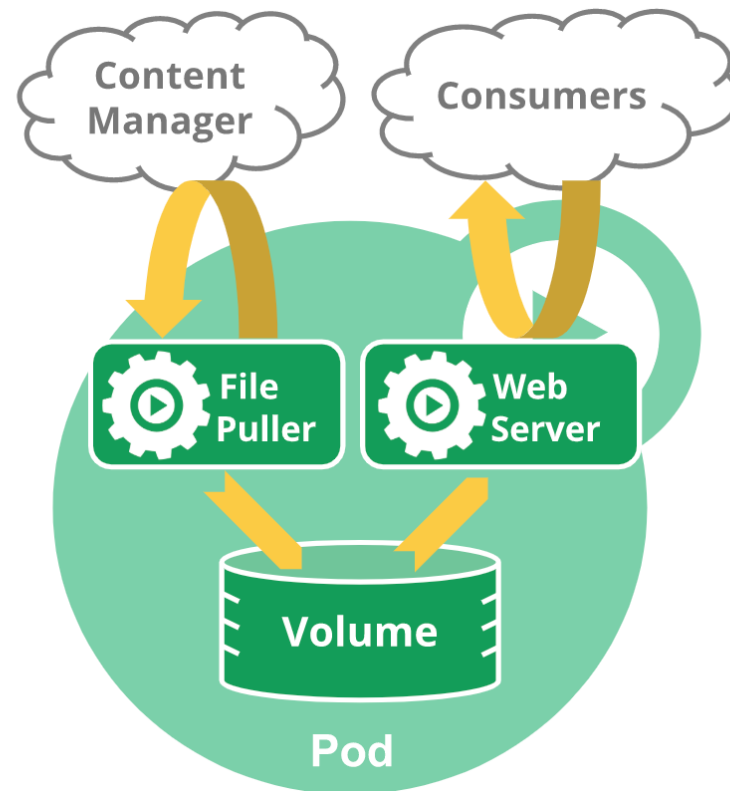
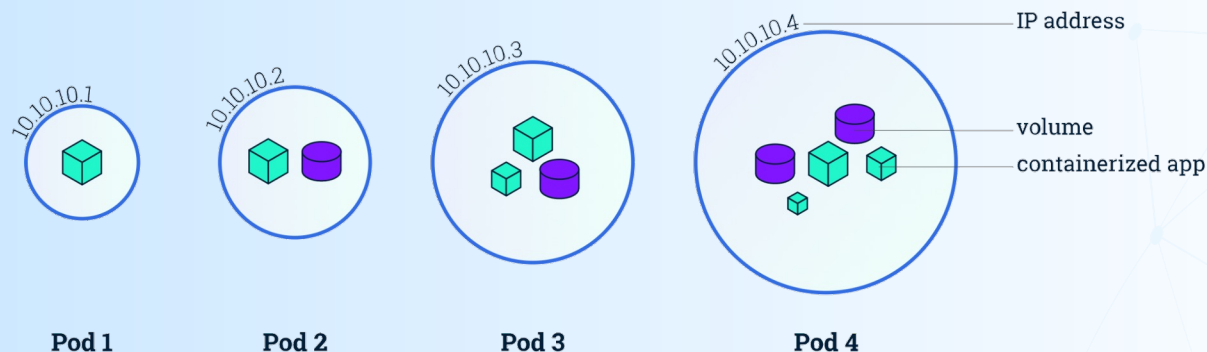
Kubernetes 核心对象简介

Kubernetes 架构



什么是 Pod ?

- Pod 只是一个逻辑概念!
- Pod 里的所有容器，共享的是同一个 Network Namespace，并且可以声明共享同一个 Volume。
- 对于 Pod 里的容器 A 和容器 B 来说：
 - ✓ 它们可以直接使用 localhost 进行通信;
 - ✓ 一个 Pod 只有一个 IP 地址，也就是这个 Pod 的 Network Namespace 对应的 IP 地址;



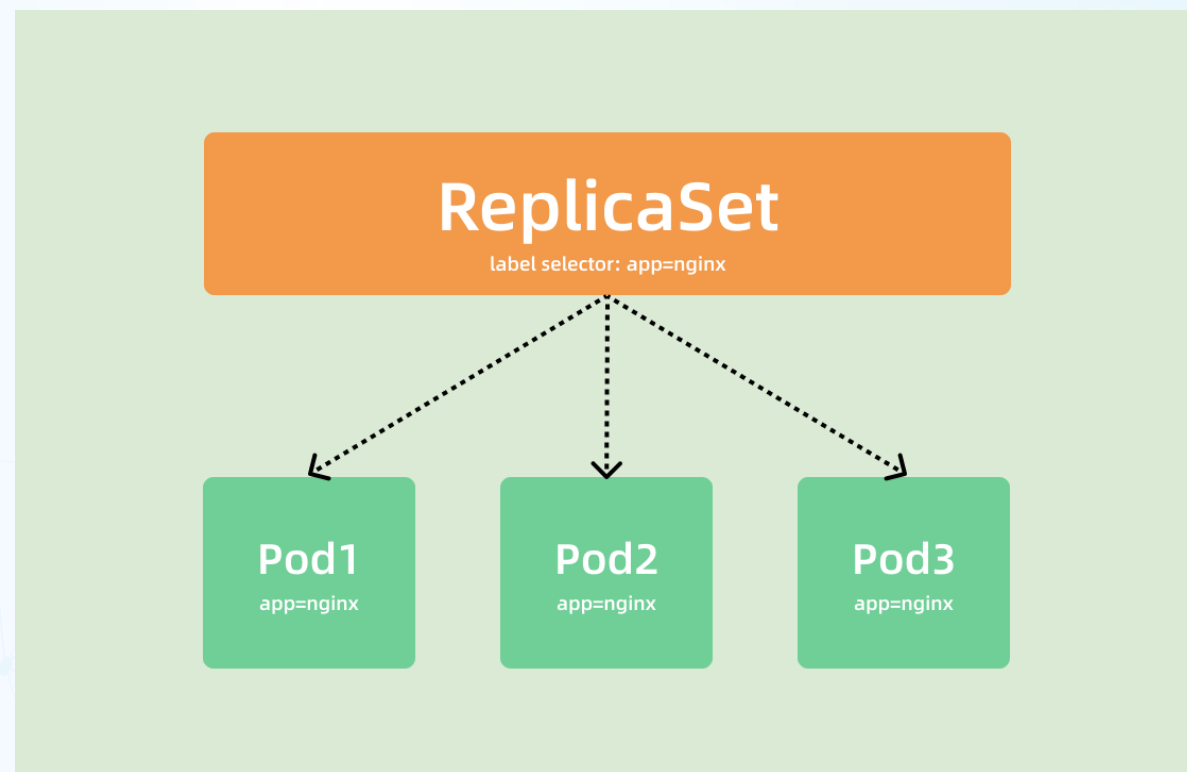
- 注:
- Pod 存在的主要目的为: 1. 抽象**进程组**概念 2. 引入**容器设计模式**
 - 一般常见都是一个应用一个pod, 多容器一般适用于sidecar场景, 比如日志收集, 网络转发等

• ReplicaSet

Pod 作为 Kubernetes 中调度运行的最小单元，在实际使用时通常并不直接创建 Pod 使用，因为 Pod 本身并不具备故障自愈的能力（即在出现故障或者异常挂掉后不能自动恢复），而作为一个成熟的编排系统来说，故障自愈能力尤为关键。

因此，Kubernetes 设计了多种工作负载用来满足各种实际需求场景，通过控制器模式对 Pod 进行故障自愈管理。

而 ReplicaSet 就是其中一种最为简单的用来管理一组 Pod 的工作负载类型（**主要用于无状态负载**），其通过标签选择器关联出一组 Pod，并对这些 Pod 的健康状态进行监控和管理，在 Pod 的 Running 数目不满足预期时，尽可能的拉起相应的 Pod 使得满足期望的运行数量。同时也可以通过对 replicas 的修改来实现 Pod 扩缩容的能力。

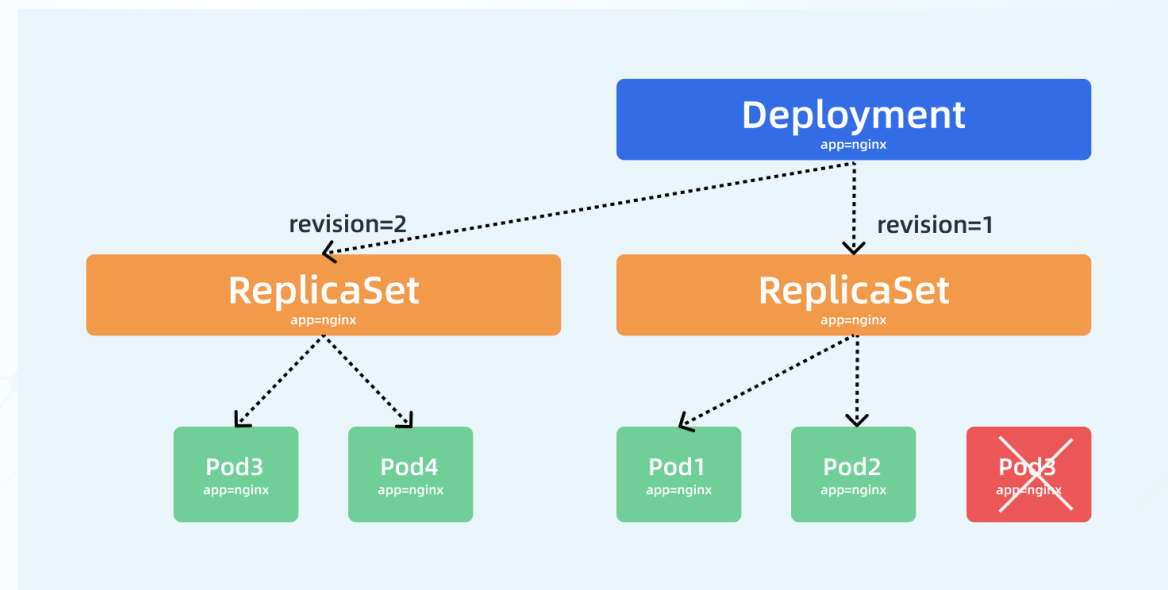


• Deployment

在实际的生产环境中 ReplicaSet 也很少直接使用，而被用于运行无状态工作负载使用最多的是 Deployment，比 ReplicaSet 更加高级的工作负载类型。

因为 Deployment 实际是通过管理 ReplicaSet 来提供和 ReplicaSet 完全一样的功能，并且在此基础上通过对于多 ReplicaSet 的管理实现了滚动更新能力。从和 ReplicaSet 资源定义上来看，其主要多出了如下关键字段：

- **Strategy**: 更新策略，支持 Recreate 和 RollingUpdate 两种策略。并且通过定义 MaxSurge 和 MaxUnavailable 来设置在更新过程中最多可多出来的 Pod 数目以及最大不可用的 Pod 数目，以此来保证在更新过程中 Pod 对外服务是尽可能可用的；
- **RevisionHistoryLimit**: 保留更新历史数目限制，由于 Deployment 管理多 ReplicaSet，因此每次更新实际都是新建了一个 ReplicaSet，并将老 ReplicaSet 缩容，新 ReplicaSet 扩容的一个过程。这样便可以随时进行回滚到历史版本，此处的历史限制决定了可以回滚到哪些版本。



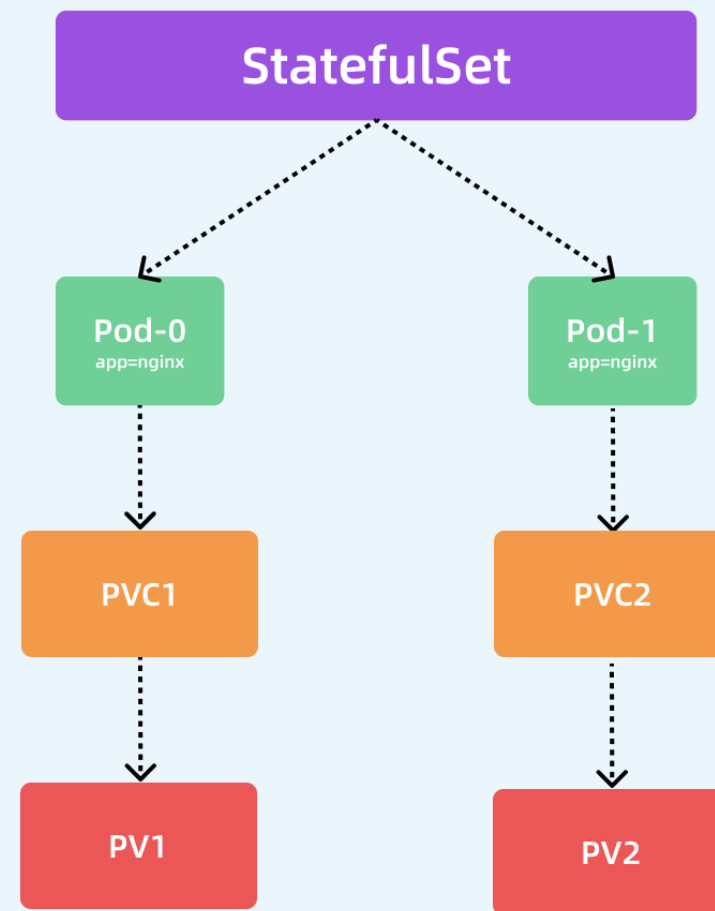
• StatefulSet

我们在前文中描述的 ReplicaSet 和 Deployment 主要用于无状态工作负载的场景（即不需要将数据持久化到本地的应用，比如web应用），而在面对有状态工作负载的场景（即需要将数据持久化到本地，比如MySQL数据库），Kubernetes设计实现了 StatefulSet 用于该需求。StatefulSet 具有以下特性：

- 稳定的、唯一的网络标识符（按照负载名称加序号的方式命名，比如sts-0）
- 稳定的、持久化的存储（volumes处定义PVC并进行动态创建）
- 有序的、优雅的部署和缩放（正序扩容Pod，倒序缩容Pod）
- 有序的、优雅的删除和终止（正序创建Pod，倒序删除Pod）
- 有序的、自动滚动更新（OnDelete、RollingUpdate）

除了有序的管理 Pod 创建、删除、扩缩外，还可以以并行的方式进行操作，可以根据不同的应用需求进行设置。

除此之外，StatefulSet 控制器还会为每个工作负载都创建一个 Headless Service，用于方便通过 Service Name 直接解析出 Pod 的 IP 列表，而不是解析出一个虚拟 IP 再由 iptables 或者 ipvs 规则转发到 Pod 中。

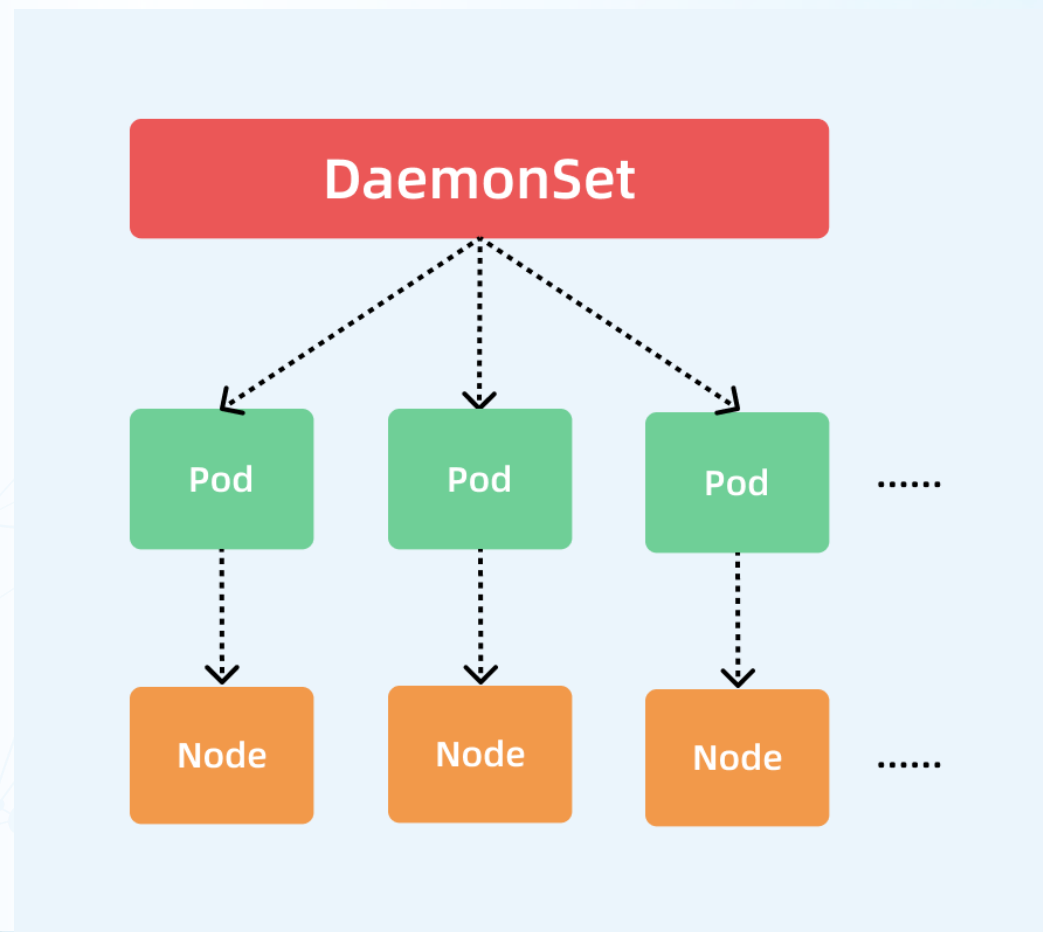


- **DaemonSet**

在某些需求场景中希望每个节点都部署一个守护进程来完成节点管理和监控采集等功能，比如我们熟知的 Flannel、Prometheus 的 Node Exporter 以及日志采集的 Fluentd 等。Kubernetes 为此设计实现了 DaemonSet 工作负载类型来满足这类需求。

DaemonSet 类型工作负载并不需要经过调度器调度，因为其在创建 Pod 时直接设置了 nodeName 字段，使得调度器不会经过任何调度策略处理；另外针对有污点的 Node，如果 Pod 没有设置任何容忍则默认不会进行部署，而不是向其他类型的工作负载会创建 Pod，但是因为调度条件不满足处于 Pending 状态。

- **UpdateStrategy**: 更新策略，支持 OnDelete 和 RollingUpdate 两种策略。由于 DaemonSet 的 Pod 不可能超出 Node 数目，所以其未定义 MaxSurge 字段，而是定义了 MaxUnavailable 来设置在更新过程中最大不可用的 Pod 数目；
- **RevisionHistoryLimit**: 保留的更新历史数目限制，默认值为 10。

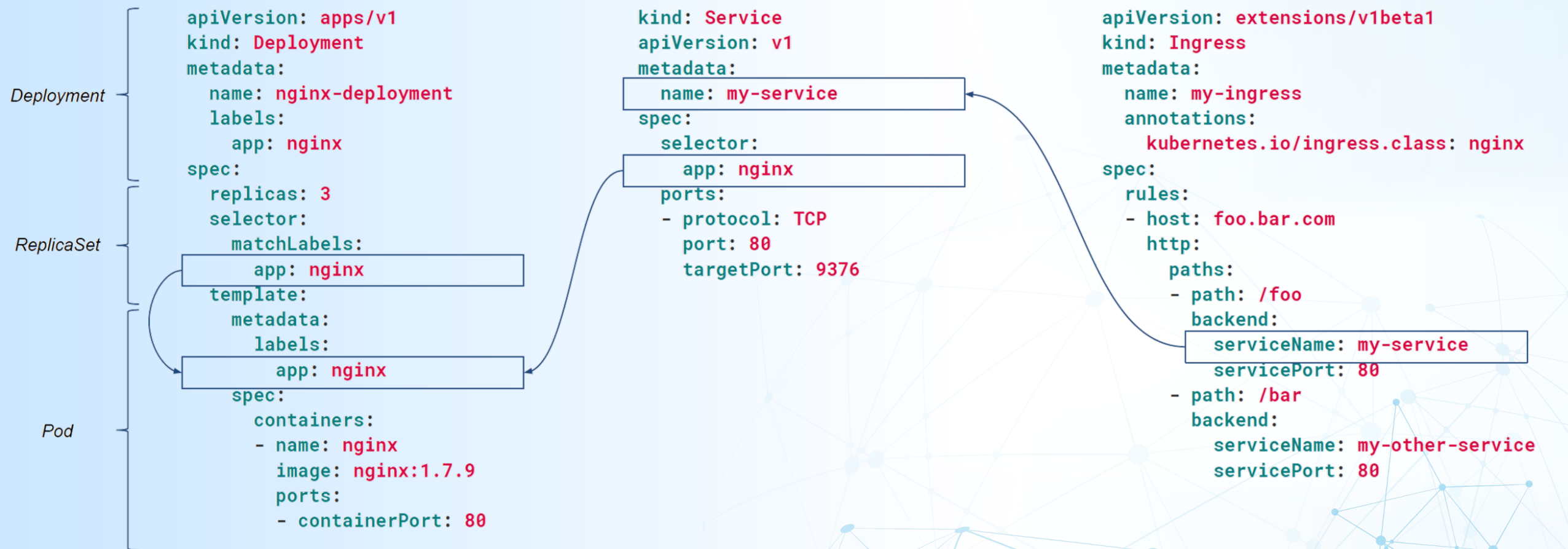


一个例子

Deployment configuration:

Service configuration:

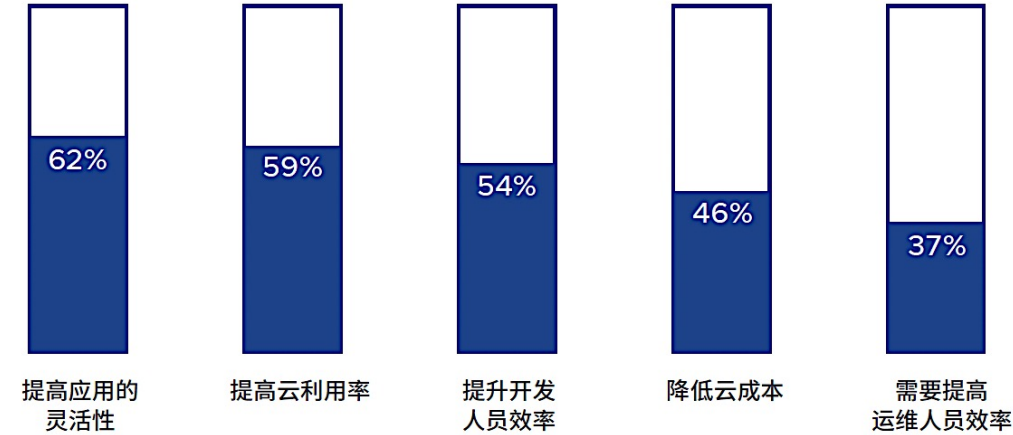
Ingress configuration:



kubectl apply -f <https://k8s.io/examples/application/deployment.yaml>
kubectl apply -f <https://k8s.io/examples/application/deployment-update.yaml>

Kubernetes 的使用越来越广泛

推动 Kubernetes 采用的主要因素



运行 Kubernetes 带来的优势



在生产环境中运行 Kubernetes 时最重要的工具或功能

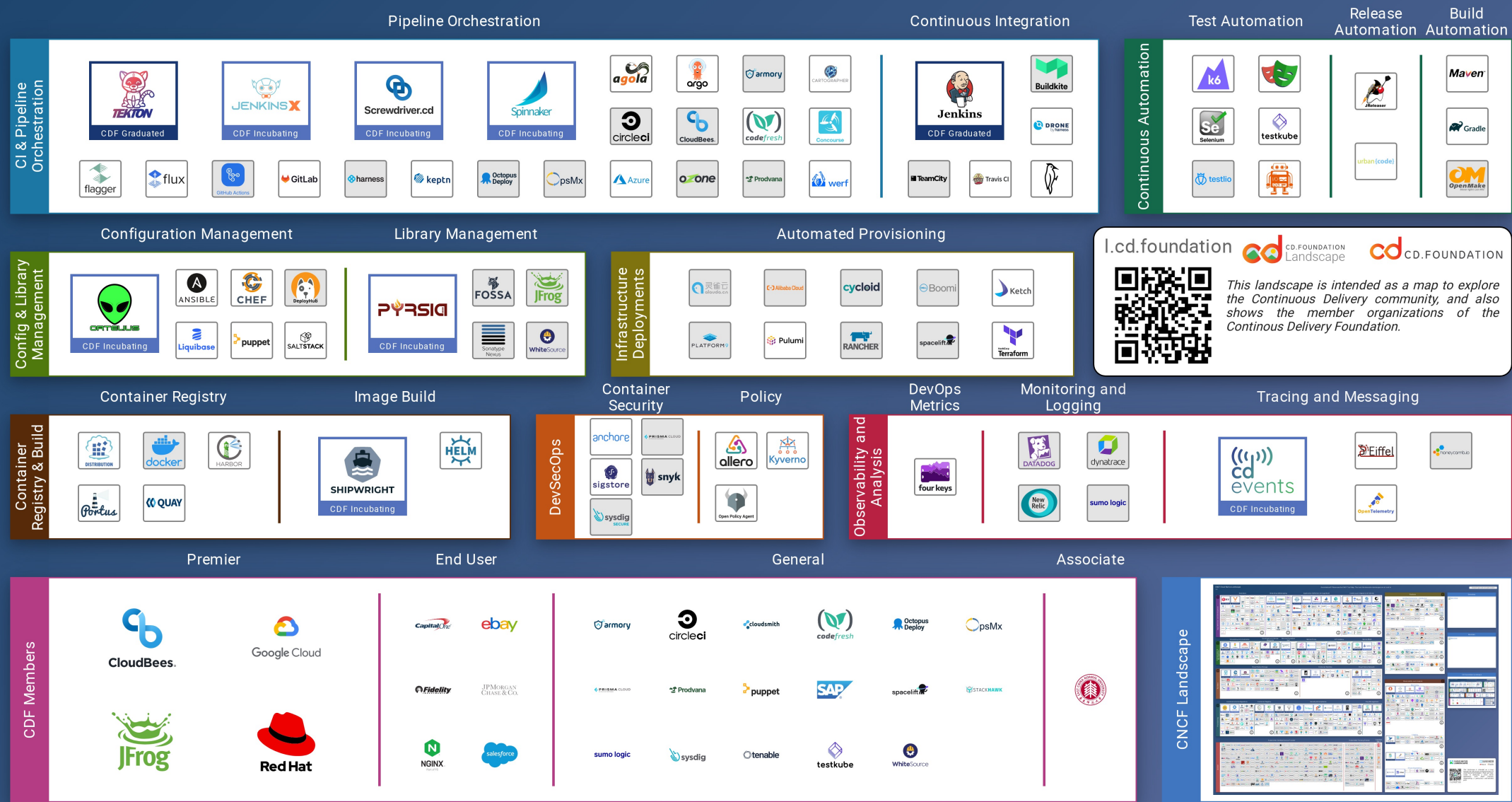


Kubernetes CI/CD 生态

Linux Foundation CI/CD Landscape
1.0

See the interactive landscape at l.cd.foundation

Greyed logos are not open source



03

DevOps 通用流程

构建明日数据世界

www.transwarp.cn

Copyright©2023 Transwarp. All Rights Reserved.

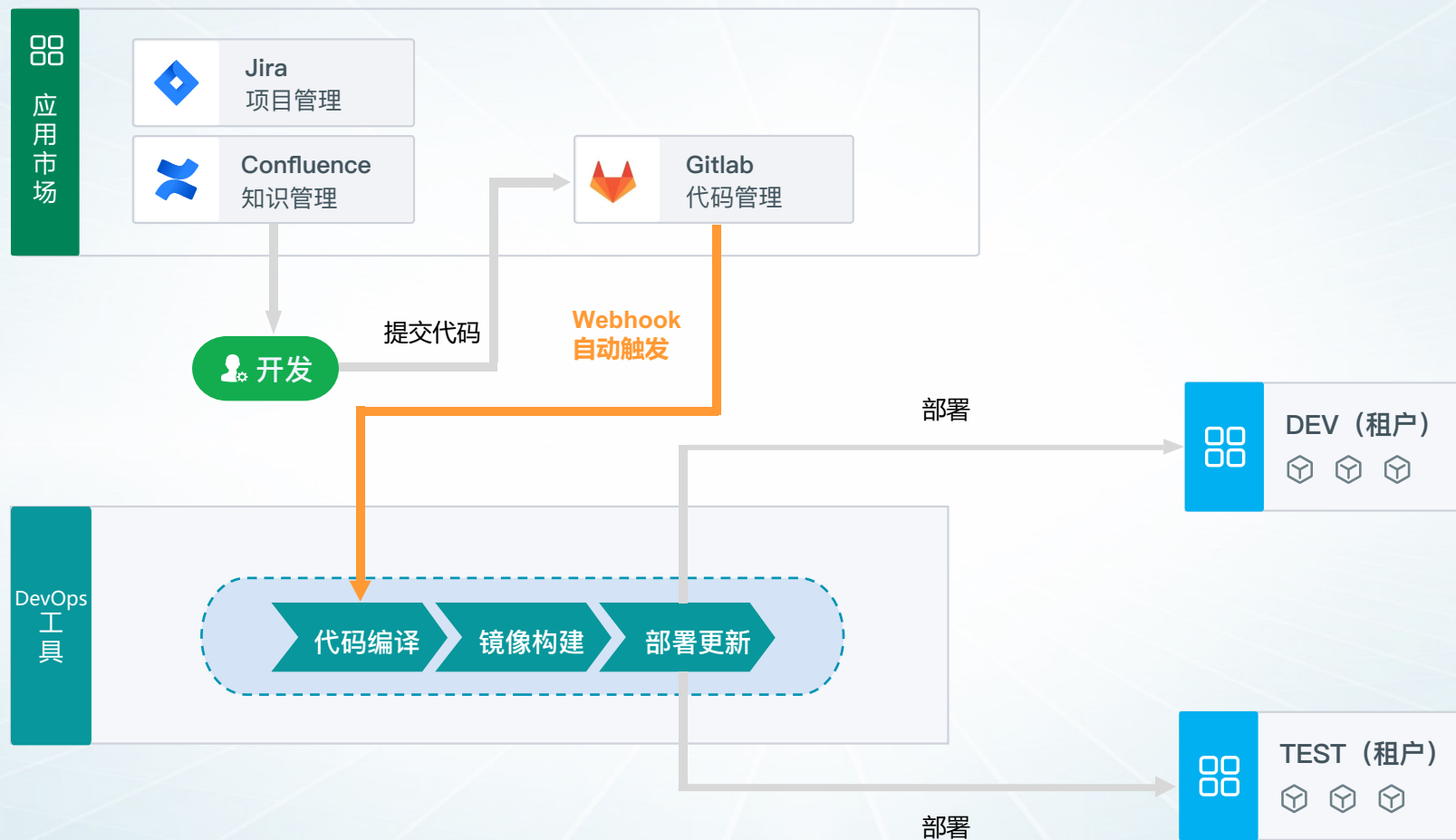
DevOps 通用流程



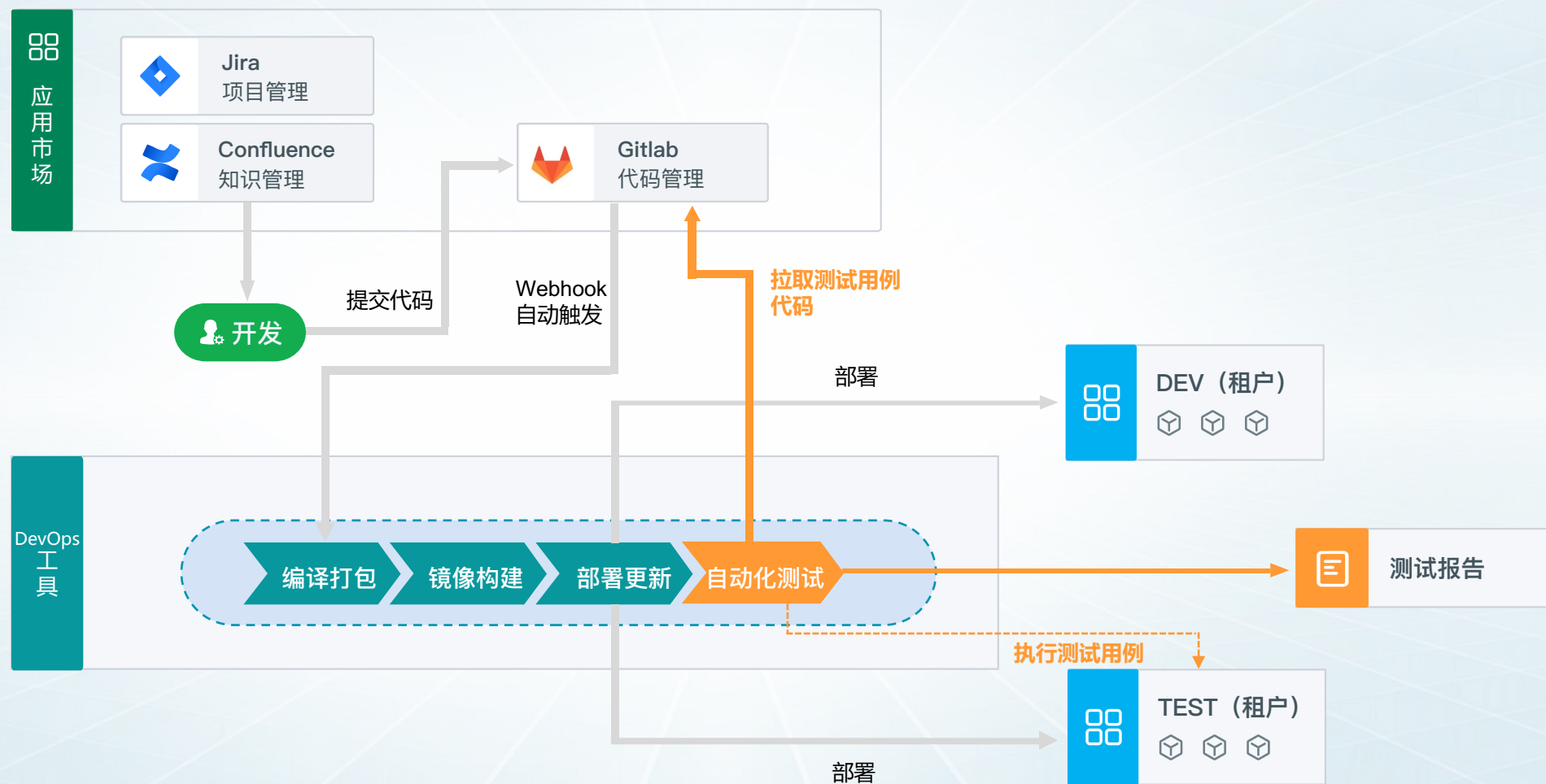
场景 1 - 对接代码仓库，实现代码自动构建部署



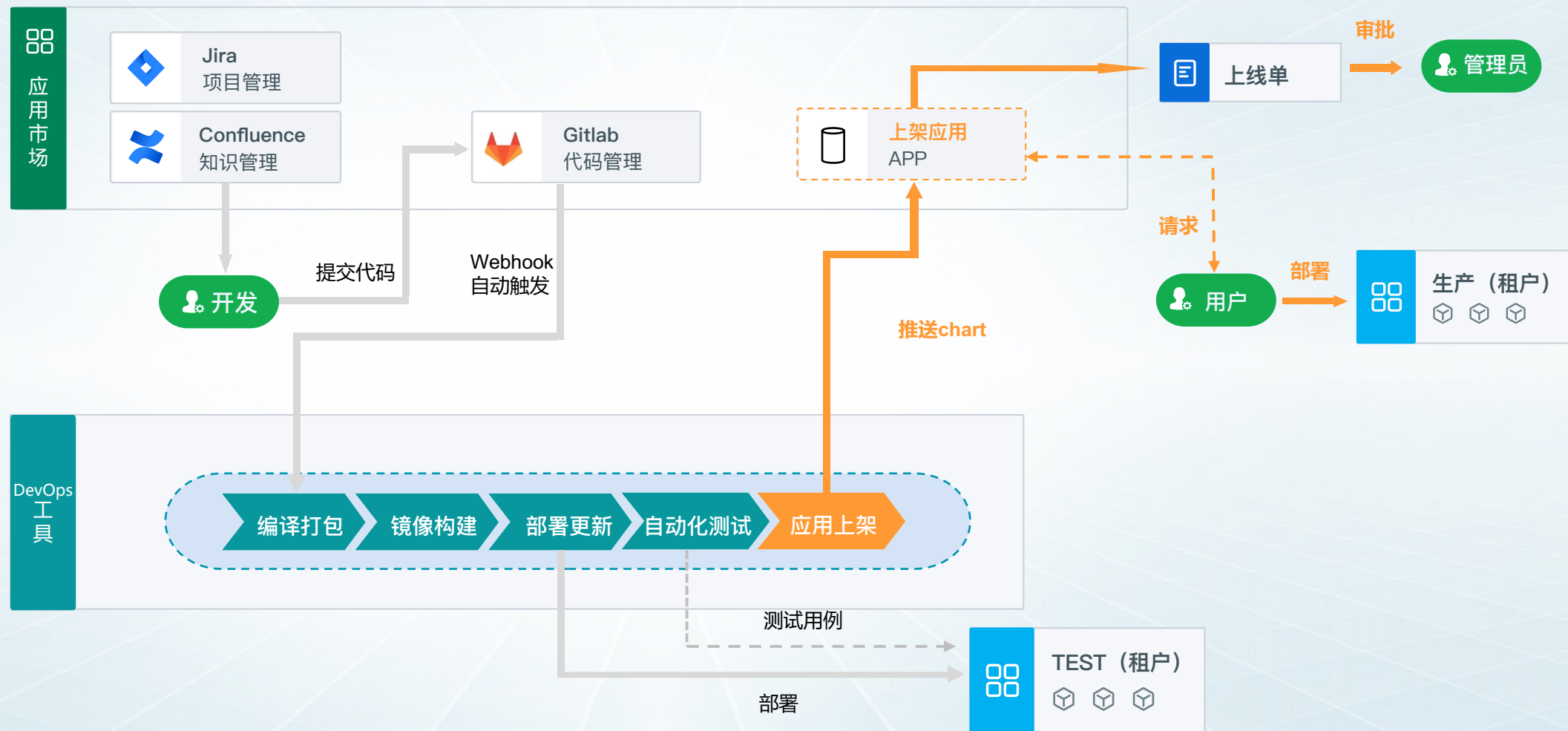
场景 2 - 提交代码自动触发流水线



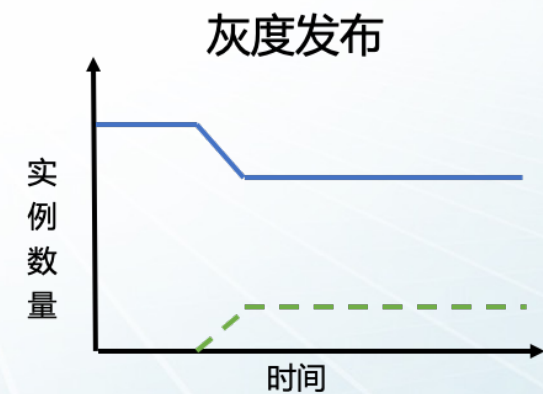
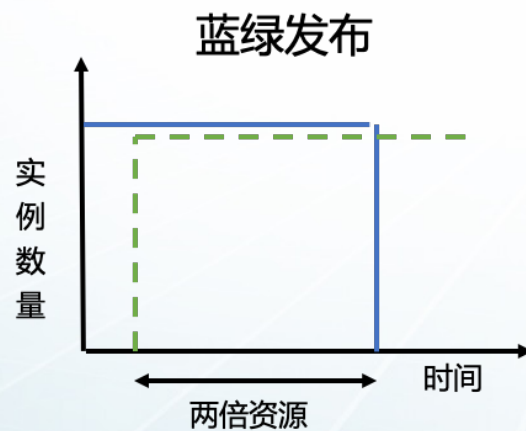
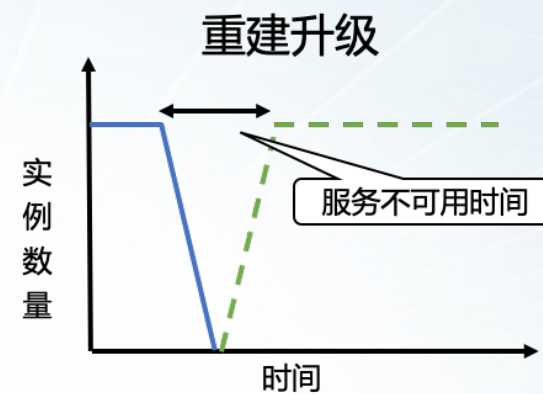
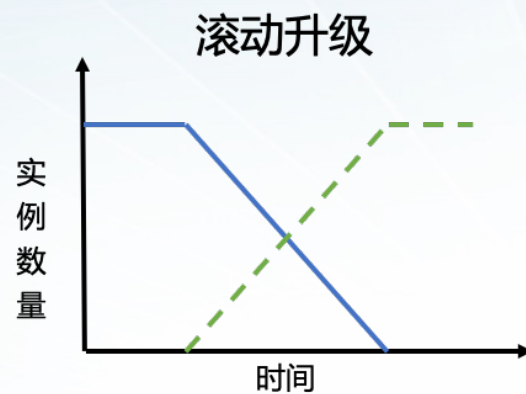
场景 3 - 自动化测试



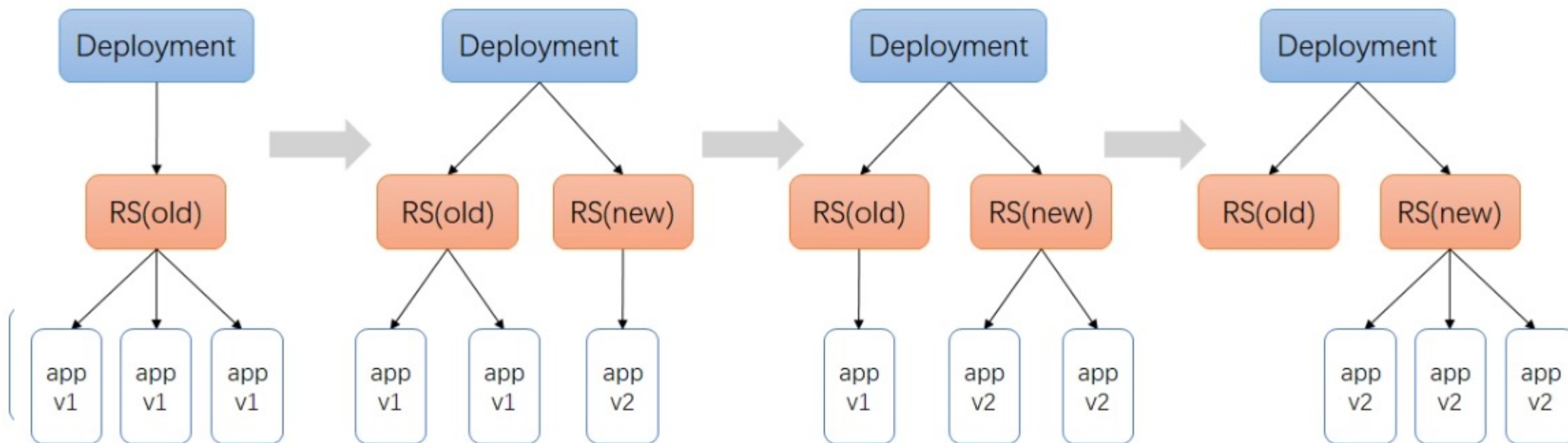
场景 4 - 应用上架



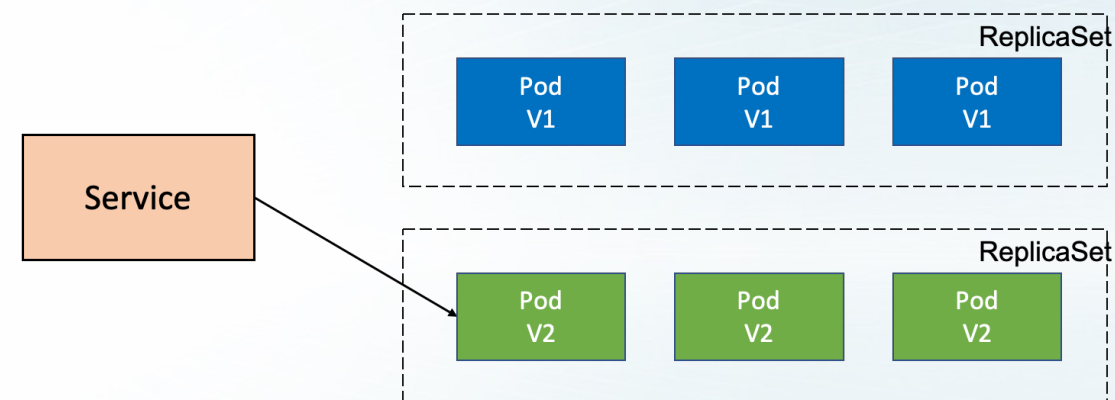
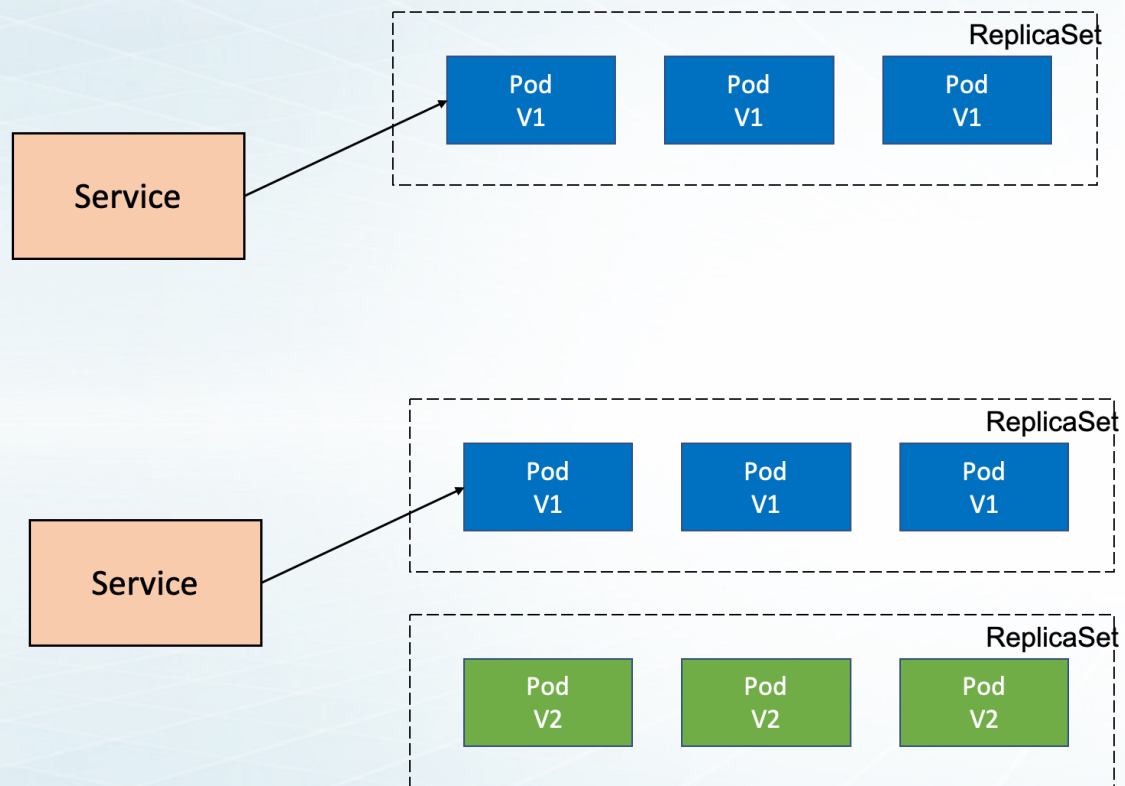
场景 5 – 应用发布策略



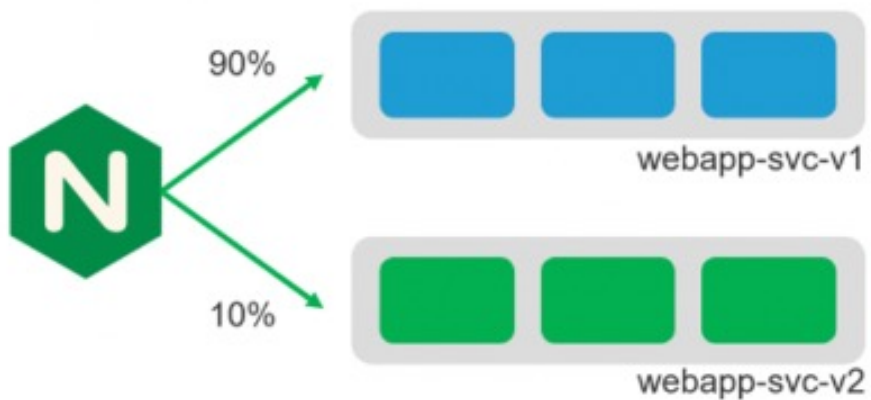
应用发布策略—滚动升级



应用发布策略—蓝绿发布



应用发布策略—灰度发布（金丝雀发布）



upstreams:

- name: webapp-v1
service: webapp-svc-v1
port: 80
- name: webapp-v2
service: webapp-svc-v2
port: 80

routes:

- path: /

splits:

- | |
|--|
| - weight: 90
action:
pass: webapp-v1 |
| - weight: 10
action:
pass: webapp-v2 |

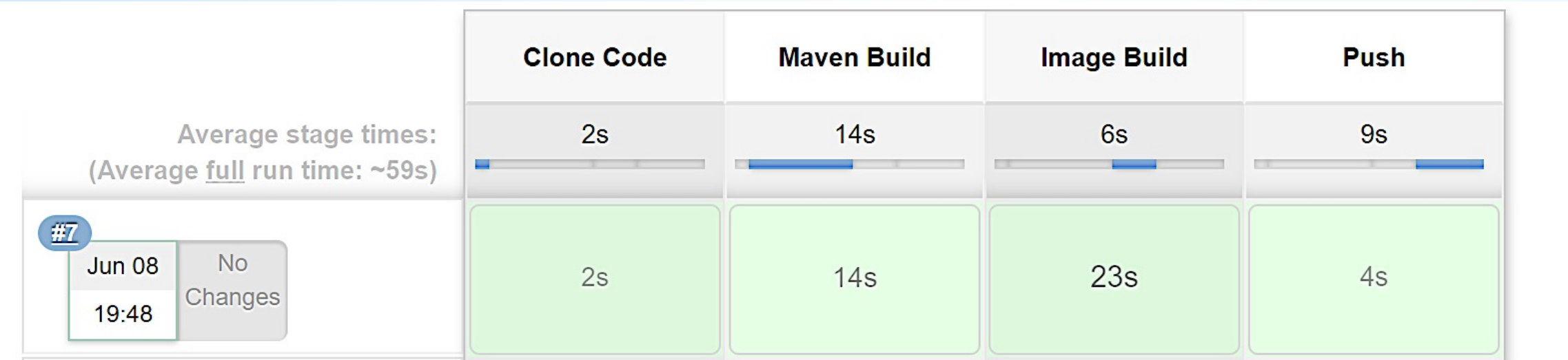
04

使用 Kubernetes 实现 DevOps 通 用流程

构建明日数据世界

www.transwarp.cn

Copyright©2023 Transwarp. All Rights Reserved.



```
pipeline {
    agent {
        label 'devops-jenkins-jenkins-slave '
    }
    // 下载代码并copy到本地
    stages {
        stage('Clone Code') {
            steps {
                echo "1.Git Clone Code"
                sh 'git clone --depth 1 --single-
branch http://root:12345678@172.16.124.11:31607/tdc/demo.git .'
            }
        }
        // 编译打包
        stage('Maven Build') {
            steps {
                echo "2.Maven Build Stage"
                sh 'mvn -B clean package -Dmaven.test.skip=true'
            }
        }
    }
}
```



```
// docker镜像构建
stage('Image Build') {
    steps {
        echo "3.Image Build Stage"
        sh 'docker build -f Dockerfile --build-arg jar_name=target/spring-boot-demo-0.0.1-SNAPSHOT.jar -t spring-boot-demo:${VERSION} .'
        sh 'docker tag spring-boot-demo:${VERSION} 172.16.124.11:30003/stage/spring-boot-demo:${VERSION}'
    }
}

//推送到tdc-harbor仓库
stage('Push') {
    steps {
        echo "4.Push Docker Image Stage"
        sh "docker login --username=admin 172.16.124.11:30003 -p Harbor12345"
        sh "docker push 172.16.124.11:30003/stage/spring-boot-demo:${VERSION}"
    }
}
}
```

```
stage('部署到k8s平台'){
  steps {
    configFileProvider([configFile(fileId: "${k8s_auth}", targetLocation:
'admin.kubeconfig')]) {
      sh """
        kubectl apply -f deploy.yaml -n ${Namespace} --kubeconfig=admin.kubeconfig
        sleep 10
        kubectl get pod -n ${Namespace} --kubeconfig=admin.kubeconfig
      """
    }
  }
}
```

部署文件 yaml 说明

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app.kubernetes.io/name: devops-njudemo
  name: springboot-njudemo
  namespace: nju
spec:
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: devops-njudemo
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      labels:
        app.kubernetes.io/name: devops-njudemo
```

```
spec:
  containers:
    - command:
      - java
      - -jar
      - /usr/lib/spring-boot-demo/spring-boot-demo.jar
      image: 172.16.124.11:30003/stage/spring-boot-demo:1.0.0
      imagePullPolicy: Always
      name: nju
      resources:
        limits:
          cpu: "1"
          memory: 1Gi
        requests:
          cpu: "1"
          memory: 1Gi
      dnsPolicy: ClusterFirst
      restartPolicy: Always
```

```
apiVersion: v1
kind: Service
metadata:
  labels:
    app.kubernetes.io/name: devops-njudemo
    app.kubernetes.io/service-type: nodeport-service
name: springboot-njudemo-svc
namespace: nju
spec:
  ports:
    - name: njudemo-port-8011
      nodePort: 31756
      port: 8011
      protocol: TCP
      targetPort: 8011
  selector:
    app.kubernetes.io/name: devops-njudemo
  sessionAffinity: None
  type: NodePort
```

```
spec:
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: devops-njudemo
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
      type: RollingUpdate
  template:
    metadata:
      labels:
        app.kubernetes.io/name: devops-njudemo
    spec:
      containers:
        - command:
            - java
            - -jar
            - /usr/lib/spring-boot-demo/spring-boot-demo.jar
          image: 172.16.124.11:30003/stage/spring-boot-demo:1.0.0
          imagePullPolicy: Always
```

replicas: 调整副本数，副本超过 1 时滚动升级才有意义

Rolling Update: 让 deployment 支持滚动更新

- **maxSurge**: 最大额外可以存在的副本数
- **maxUnavailable**: 最大不可用状态的 Pod 的最大值

这两个参数的默认值都是 25%，如果我们更新一个 100 Pod 的 Deployment，会立刻创建 25 个新 pod，同时会关闭 25 个旧 Pod

image: 更换 image 实现应用版本升级

springboot-njudemo-6848766d9c-jjn4v	0/1	ContainerCreating	0	2s
springboot-njudemo-76b97f8f8c-6w46l	1/1	Running	0	45m
springboot-njudemo-76b97f8f8c-cj5gk	1/1	Terminating	0	22m
springboot-njudemo-76b97f8f8c-m9s8q	1/1	Running	0	22m
springboot-njudemo-76b97f8f8c-xhxl2	1/1	Running	0	22m

05

演示

构建明日数据世界

www.transwarp.cn

Copyright©2023 Transwarp. All Rights Reserved.

THANKS

星环科技致力成为世界领先的基础软件供应商

T R A N S W A R P

星环信息科技（上海）股份有限公司

www.transwarp.cn

Copyright©2023 Transwarp. All Rights Reserved.